

Extend（扩展）文件夹详解

Extend 文件夹包含项目中常用的 **分类扩展（Category）**，基于 Objective-C 的 Runtime 机制，为系统类（如 NSDate、UIColor、UIButton）添加实用功能，避免重复开发，提升代码复用率。以下逐一讲解各分类的功能、使用方式及优化建议：

一、NSDate+Exts - 日期处理神器

1. 核心功能

为 NSDate 类扩展日期与字符串转换、日期计算、日期比较等功能，解决 iOS 原生日期处理繁琐的问题：

- 字符串与日期互转（支持自定义格式，如 yyyy-MM-dd、yyyy-MM-dd HH:mm:ss）；
- 日期差值计算（如两个日期相差天数、小时数）；
- 友好日期显示（如“刚刚”“3 分钟前”“昨天”）。

2. 核心实现代码

```
// 1. 定义常用日期格式宏（头文件中）
#define NSDATE_FORMAT_NORMAL @"yyyy-MM-dd HH:mm:ss" // 完整时间格式
#define NSDATE_FORMAT_DATE @"yyyy-MM-dd" // 仅日期格式
#define LOCALE_CHINA [[NSLocale alloc] initWithLocaleIdentifier:@"zh_CN"] // 中文Locale

// 2. 字符串转日期（类方法）
+ (id)dateWithString:(NSString *)string format:(NSString *)format {
    if (!string || !format) return nil;

    NSDateFormatter *formatter = [[NSDateFormatter alloc] init];
    formatter.dateFormat = format;
    formatter.locale = LOCALE_CHINA; // 解决不同地区时间格式差异

    return [formatter dateFromString:string];
}

// 3. 日期转字符串（实例方法）
```

```
- (NSString *)stringWithFormat:(NSString *)format {
    if (!format) return @"";

    NSDateFormatter *formatter = [[NSDateFormatter alloc] init];
    formatter.dateFormat = format;
    formatter.locale = LOCALE_CHINA;

    return [formatter stringFromDate:self];
}
```

3. 使用场景示例

```
// 1. 服务器时间字符串转 NSDate
NSString *serverTimeStr = @"2024-05-20 14:30:00";
NSDate *serverDate = [NSDate dateWithString:serverTimeStr format:NSDATE_FORMAT_
NORMAL];
// 2. NSDate 转显示用字符串（仅日期）
NSString *showDateStr = [serverDate stringWithFormat:NSDATE_FORMAT_DATE]; // 输出：20
24-05-20
// 3. 计算与当前日期的差值（相差天数）
NSInteger daysDiff = [self daysSinceDate:serverDate]; // 自定义差值计算方法
NSLog(@"与服务日期相差 %ld 天", daysDiff);
// 4. 显示友好日期（如“3分钟前”）
NSString *friendlyDate = [serverDate friendlyDisplay]; // 需自定义实现，根据时间差返回对应字
符串
```

4. 优化建议

(1) 缓存 NSDateFormatter，提升性能

NSDateFormatter 创建成本高，频繁创建会影响性能，建议用单例缓存：

```
// 新增静态字典缓存 Formatter
static NSMutableDictionary *s_formatterCache;
+ (void)load {
    s_formatterCache = [NSMutableDictionary dictionary];
}
```

```

// 优化后的字符串转日期方法
+ (id)dateWithString:(NSString *)string format:(NSString *)format {
    if (!string || !format) return nil;

    // 从缓存获取 Formatter，无则创建
    NSDateFormatter *formatter = s_formatterCache[format];
    if (!formatter) {
        formatter = [[NSDateFormatter alloc] init];
        formatter.dateFormat = format;
        formatter.locale = LOCALE_CHINA;
        s_formatterCache[format] = formatter; // 存入缓存
    }

    return [formatter dateFromString:string];
}

```

(2) 处理多线程安全问题

`NSDateFormatter` 非线程安全，多线程环境下需加锁：

```

// 新增静态锁
static dispatch_semaphore_t s_formatterLock;
+ (void)load {
    s_formatterCache = [NSMutableDictionary dictionary];
    s_formatterLock = dispatch_semaphore_create(1); // 初始化信号量锁
}
// 加锁后的方法
+ (id)dateWithString:(NSString *)string format:(NSString *)format {
    if (!string || !format) return nil;

    dispatch_semaphore_wait(s_formatterLock, DISPATCH_TIME_FOREVER); // 加锁
    NSDateFormatter *formatter = s_formatterCache[format];
    if (!formatter) {
        formatter = [[NSDateFormatter alloc] init];
        formatter.dateFormat = format;
        formatter.locale = LOCALE_CHINA;
        s_formatterCache[format] = formatter;
    }
    NSDate *date = [formatter dateFromString:string];
    dispatch_semaphore_signal(s_formatterLock); // 解锁
}

```

```
    return date;
}
```

二、UIColor+Hex - 颜色处理小助手

1. 核心功能

为 UIColor 类扩展 **十六进制字符串转颜色** 的功能，替代原生的 RGB 数值计算，更符合设计稿习惯（设计稿通常提供十六进制颜色值）。

2. 核心实现代码

```
// 1. 十六进制字符串转颜色（支持 #/0X 前缀或无前缀）
+ (UIColor *)colorWithHexString:(NSString *)color {
    // 1. 清理字符串（去除空格、换行，转为大写）
    NSString *cString = [[color stringByTrimmingCharactersInSet:[NSCharacterSet
    whitespaceAndNewlineCharacterSet]] uppercaseString];

    // 2. 处理前缀（# 或 0X）
    if ([cString hasPrefix:@"0X"]) {
        cString = [cString substringFromIndex:2];
    }
    if ([cString hasPrefix:@"#"]) {
        cString = [cString substringFromIndex:1];
    }

    // 3. 校验字符串长度（必须为6位，如 123456）
    if (cString.length != 6) {
        return [UIColor clearColor]; // 非法格式返回透明色
    }

    // 4. 解析 RGB 分量
    NSString *rString = [cString substringWithRange:NSMakeRange(0, 2)]; // 红色分量（前2位）
    NSString *gString = [cString substringWithRange:NSMakeRange(2, 2)]; // 绿色分量（中间2位）
    NSString *bString = [cString substringWithRange:NSMakeRange(4, 2)]; // 蓝色分量（后2位）
}
```

```

// 5. 十六进制转十进制
unsigned int r, g, b;
[[NSScanner scannerWithString:rString] scanHexInt:&r];
[[NSScanner scannerWithString:gString] scanHexInt:&g];
[[NSScanner scannerWithString:bString] scanHexInt:&b];

// 6. 创建并返回颜色（RGB 范围 0~1，需除以 255）
return [UIColor colorWithRed:((float)r / 255.0f)
                        green:((float)g / 255.0f)
                        blue:((float)b / 255.0f)
                        alpha:1.0f];
}

// 2. 带透明度的十六进制颜色（扩展方法）
+ (UIColor *)colorWithHexString:(NSString *)color withAlpha:(CGFloat)alpha {
    UIColor *rgbColor = [self colorWithHexString:color];
    return [rgbColor colorWithAlphaComponent:alpha];
}

```

3. 使用示例

```

// 1. 无前缀十六进制字符串
self.view.backgroundColor = [UIColor colorWithHexString:@"123456"];
// 2. 带 # 前缀
self.titleLabel.textColor = [UIColor colorWithHexString:@"#FFFFFF"];
// 3. 带 0X 前缀
self.button.backgroundColor = [UIColor colorWithHexString:@"0XFF5733"];
// 4. 带透明度（0.5）
self.layer.backgroundColor = [UIColor colorWithHexString:@"123456" withAlpha:0.5].CGColor
;

```

4. 优化建议

（1）添加颜色缓存，避免重复创建

相同十六进制字符串无需重复解析，用字典缓存已创建的颜色：

```

// 静态缓存字典（键：十六进制字符串+透明度，值：UIColor）
static NSMutableDictionary *s_colorCache;
+ (void)load {
    s_colorCache = [NSMutableDictionary dictionary];
}
// 优化后的带透明度方法
+ (UIColor *)colorWithHexString:(NSString *)color withAlpha:(CGFloat)alpha {
    // 生成缓存键（包含透明度，避免相同颜色不同透明度冲突）
    NSString *cacheKey = [NSString stringWithFormat:@"% %@%.1f", color, alpha];
    UIColor *cachedColor = s_colorCache[cacheKey];
    if (cachedColor) {
        return cachedColor;
    }

    // 解析颜色（原有逻辑）
    UIColor *rgbColor = [self colorWithHexString:color];
    UIColor *resultColor = [rgbColor colorWithAlphaComponent:alpha];

    // 存入缓存
    s_colorCache[cacheKey] = resultColor;
    return resultColor;
}

```

（2）增强格式校验

处理非法字符串（如长度不足 6 位、包含非十六进制字符），返回默认颜色：

```

+ (UIColor *)colorWithHexString:(NSString *)color {
    // 原有清理逻辑...

    // 新增：校验是否为十六进制字符
    NSCharacterSet *hexSet = [NSCharacterSet characterSetWithCharactersInString:@"0123456789ABCDEF"];
    for (NSUInteger i = 0; i < cString.length; i++) {
        unichar ch = [cString characterAtIndex:i];
        if (![hexSet characterIsMember:ch]) {
            return [UIColor lightGrayColor]; // 非法字符返回浅灰色
        }
    }
}

```

```
// 原有解析逻辑...  
}
```

三、UIButton+Badge - 角标提醒扩展

1. 核心功能

为 UIButton 类动态添加 **数字角标** 功能，用于消息未读提醒、通知数量显示等场景，支持自定义角标样式（背景色、文字色、大小、位置）。

2. 核心实现代码

```
// 1. 用 Runtime 动态添加角标相关属性（头文件中声明）  
@interface UIButton (Badge)  
@property (nonatomic, copy) NSString *badgeValue;    // 角标数字（如 @"3"）  
@property (nonatomic, strong) UIColor *badgeBGColor; // 角标背景色（默认红色）  
@property (nonatomic, strong) UIColor *badgeTextColor; // 角标文字色（默认白色）  
@property (nonatomic, strong) UIFont *badgeFont;    // 角标字体（默认12号）  
@property (nonatomic, assign) CGFloat badgePadding;  // 角标内边距（默认6px）  
@property (nonatomic, assign) CGFloat badgeMinSize; // 角标最小尺寸（默认8px）  
@property (nonatomic, assign) BOOL shouldHideBadgeAtZero; // 数字为0时隐藏（默认YES）  
@property (nonatomic, assign) BOOL shouldAnimateBadge; // 数字变化时动画（默认YES）  
@end  
  
// 2. 初始化角标默认样式  
- (void)badgeInit {  
    // 默认样式  
    self.badgeBGColor = [UIColor redColor];  
    self.badgeTextColor = [UIColor whiteColor];  
    self.badgeFont = [UIFont systemFontOfSize:12.0];  
    self.badgePadding = 6;  
    self.badgeMinSize = 8;  
  
    // 默认位置（右上角，偏移-4px）  
    self.badgeOriginX = self.frame.size.width - self.badge.frame.size.width / 2;  
    self.badgeOriginY = -4;  
  
    // 默认规则
```

```

self.shouldHideBadgeAtZero = YES;
self.shouldAnimateBadge = YES;
}
// 3. 更新角标Frame (计算大小与位置)
- (void)updateBadgeFrame {
    // 1. 计算角标文字尺寸
    CGSize expectedLabelSize = [self.badgeValue sizeWithAttributes:@{NSFontAttributeName:
self.badgeFont}];

    // 2. 确保角标为圆形 (最小宽高取最大值)
    CGFloat minHeight = (expectedLabelSize.height < self.badgeMinSize) ? self.badgeMinSize :
expectedLabelSize.height;
    CGFloat minWidth = (expectedLabelSize.width < minHeight) ? minHeight : expectedLabelSize
.width;

    // 3. 设置角标Frame (包含内边距)
    self.badge.frame = CGRectMake(
        self.badgeOriginX,
        self.badgeOriginY,
        minWidth + self.badgePadding,
        minHeight + self.badgePadding
    );

    // 4. 设置角标为圆形 (圆角=高度/2)
    self.badge.layer.cornerRadius = (minHeight + self.badgePadding) / 2;
    self.badge.clipsToBounds = YES; // 裁剪圆角
}

```

3. 使用示例

```

// 1. 创建按钮并设置角标
UIButton *messageBtn = [UIButton buttonWithType:UIButtonTypeCustom];
messageBtn.frame = CGRectMake(100, 100, 44, 44);
[self.view addSubview:messageBtn];
// 2. 设置角标数字 (显示3条未读消息)
messageBtn.badgeValue = @"3";
// 3. 自定义角标样式
messageBtn.badgeBGColor = [UIColor orangeColor]; // 背景色改为橙色
messageBtn.badgeTextColor = [UIColor blackColor]; // 文字色改为黑色

```



```
messageBtn.badgeFont = [UIFont boldSystemFontOfSize:11]; // 字体加粗
messageBtn.shouldAnimateBadge = YES; // 数字变化时显示动画
// 4. 更新角标（如未读消息变为5条）
messageBtn.badgeValue = @"5"; // 自动触发动画与Frame更新
```

4. 优化建议

(1) 监听按钮 Frame 变化，同步更新角标位置

当按钮位置或大小变化时（如屏幕旋转、父视图布局调整），角标位置需同步更新，可通过 KVO 监听按钮 frame 属性：

```
- (void)setBadgeValue:(NSString *)badgeValue {
    // 原有逻辑...

    // 添加KVO监听按钮frame变化
    [self addObserver:self forKeyPath:@"frame" options:NSKeyValueObservingOptionNew
    context:nil];
}
// KVO回调：按钮frame变化时更新角标位置
- (void)observeValueForKeyPath:(NSString *)keyPath ofObject:(id)object change:(
NSDictionary<NSKeyValueChangeKey,id> *)change context:(void *)context {
    if ([keyPath isEqualToString:@"frame"]) {
        [self updateBadgeFrame]; // 重新计算角标位置
    }
}
// 移除KVO监听，避免内存泄漏
- (void)dealloc {
    [self removeObserver:self forKeyPath:@"frame"];
}
```

(2) 适配不同屏幕尺寸

角标位置偏移量（`badgeOriginX/badgeOriginY`）需根据按钮大小动态调整，避免在大尺寸屏幕上位置偏移：

```
// 优化角标位置计算（基于按钮宽度的比例）
- (void)updateBadgeFrame {
    // 原有逻辑...

    // 动态计算X轴偏移（按钮右侧1/4处，适配不同按钮宽度）
    self.badgeOriginX = self.frame.size.width - (minWidth + self.badgePadding) / 2 - self.frame.size
    .width * 0.1;
    // 动态计算Y轴偏移（按钮顶部1/4处）
    self.badgeOriginY = (minHeight + self.badgePadding) / 2 - self.frame.size.height * 0.1;

    // 原有Frame设置逻辑...
}
```

四、UIButton+touch - 防重复点击扩展

1. 核心功能

为 UIButton 类添加 **防重复点击** 功能，避免用户快速连续点击按钮导致的重复操作（如重复提交表单、重复跳转页面），通过方法交换（Method Swizzling）实现全局生效。

2. 核心实现代码

```
// 1. 用Runtime动态添加属性（头文件中声明）
@interface UIButton (touch)
@property (nonatomic, assign) NSTimeInterval timeInterval; // 点击间隔（默认2秒）
@property (nonatomic, assign)
```

（注：文档部分内容可能由 AI 生成）